

Question: How does this Block World Solver code move blocks from the start state to the goal state?

This Block World Solver compares each block's current position with its goal position. If a block is not in the right place, it moves the block step-by-step to the desired position. It repeats this until all blocks match the goal state.

Step-by-step Explanation of `solve_blocks` Function:

1. Print Initial State:

The function first shows where each block is placed at the start. For example, block A is on B, block B is on the Table, etc.

2. Print Goal State:

It then shows the desired final arrangement of the blocks, where each block should be at the end.

3. Copy Current State:

It makes a copy of the starting positions so it can keep track of changes without modifying the original data.

4. Compare Each Block's Position:

For every block in the goal state:

- It checks if the block is already where it should be.
- If not, it prints a step showing the block being moved from its current position to the desired position.

5. Update the Current State:

After moving a block, it updates the current positions to reflect the move.

6. Repeat Until All Blocks Are in Place:

It continues checking and moving blocks one by one until all blocks match the goal state.

7. Print Final State:

Finally, it prints the last arrangement of blocks confirming the goal is reached.

Run Code: <https://www.programiz.com/online-compiler/35xf5bBCWVXEb>

Program: main.py (without AI)

```
# Simple Block World Solver (INPUT).

start_state = {
    "A": "B",
    "B": "Table",
    "C": "Table"
}

goal_state = {
    "C": "B",
    "B": "A",
    "A": "Table"
}

.....

# A function that compares states and helps reach the goal.
def solve_blocks(start, goal):
    print(" Initial State:")
    for k, v in start.items():
        print(f"{k} is on {v}")

    print("\n🎯 Goal State:")
    for k, v in goal.items():
        print(f"{k} should be on {v}")

    print("\n🔄 Steps to reach goal:\n")

    # Copy the start state to keep track of current block positions.
    current_state = start.copy()
    steps = 1

    # Iterate over each block and its desired position in the goal state.
    for block, desired_position in goal.items():
        # If the block is not already at the desired position, move it.
        if current_state.get(block) != desired_position:
            print(f"Step {steps}: Move {block} from {current_state[block]} to {desired_position}")
            # Update the current state to reflect the move.
            current_state[block] = desired_position
            steps += 1

    print("\n🏁 Final State Reached!")

    print("\n📦 Final State:")
    for k, v in current_state.items():
        print(f"{k} is on {v}")

# Run the code.
solve_blocks(start_state, goal_state)
```

Output

Clear

■ Initial State:

A is on B
B is on Table
C is on Table

🎯 Goal State:

C should be on B
B should be on A
A should be on Table

📋 Steps to reach goal:

Step 1: Move C from Table to B
Step 2: Move B from Table to A
Step 3: Move A from B to Table

✅ Final State Reached!

🏠 Final State:

A is on Table
B is on A
C is on B